

C270 DevOps Notes - Expanded Student Version

DevNet / DevOps fundamentals, APIs, Git, Docker, Jenkins, CI/CD, troubleshooting, and command cheat sheet

How to use these notes: Read the short explanation first, then use the tables to compare ideas quickly. The command sections use placeholders like <repo-url>, <branch>, and <container-name> so you can swap in whatever your lab or exam gives you. This version has extra sentences so the ideas feel less like random keywords and more like a story your brain can actually follow.

Quick Exam Map

The DevOps topics in this module connect together like a small delivery factory. Software development explains how an app moves from idea to working code. Git keeps track of the code changes. Docker packages the app so it can run consistently. Jenkins or another CI/CD tool automates building and testing, so the team does not have to do every step manually. APIs allow apps and systems to communicate, and security makes sure that communication is controlled instead of chaotic.

Area	Big idea	Exam clue
Linux and Python	Basic tools used to navigate files, write scripts, and automate tasks.	Questions may ask what command changes folder, checks files, or runs a Python app.
SDLC and Agile	Ways to plan, build, test, deploy, and improve software.	Look for words like sprint, backlog, user story, waterfall, or lean waste.
Git and GitHub	Version control for tracking code changes and team collaboration.	Look for stage, commit, branch, merge, clone, pull, push, and conflict.
APIs and REST	A controlled way for software systems to talk to each other.	Look for HTTP methods, status codes, authentication, tokens, and rate limits.
Docker	Packages an app with dependencies into an image and runs it as a container.	Look for Dockerfile, image, container, build, run, ports, and logs.
Jenkins and CI/CD	Automates build, test, and deployment steps after code changes.	Look for pipeline, job, trigger, build stage, test stage, and deploy stage.

Module 1 - Course Introduction, Linux, and Python

Module 1 is the toolbox chapter. It does not only introduce DevNet; it also prepares you to survive the labs without getting lost in the terminal forest. In DevOps, you often work with servers, containers, cloud environments, and code repositories, so being comfortable with Linux commands is important. Python is also useful because many automation scripts and small web applications are written in Python.

Lab Environment and Virtual Machines

A virtual machine is a software computer running inside a physical computer. The physical computer is called the host, while the VM is the guest. The DEVASC VM gives students a ready-made Linux lab environment with tools already installed. This matters because everyone can use a similar setup, which reduces the “but it works on my laptop” problem.

Term	Meaning	Why it matters
Host	The real physical computer.	It provides CPU, memory, disk, and network resources.
Guest / VM	The virtual computer running on the	It gives you a safe practice

	host.	environment.
Virtualization	Technology that lets one machine run multiple virtual computers.	Useful for labs, testing, servers, and cloud platforms.
Codespaces	A browser-based development environment on GitHub.	It can replace some VM workflows by giving you a cloud dev environment.

Linux Basics You Actually Use

Linux commands are short, but they are powerful. Most DevOps work happens through a terminal because it is faster to repeat, automate, and troubleshoot. You do not need to memorize every Linux command in the universe. Focus on navigation, file checking, permissions, and running scripts because those appear again in Git, Docker, and Jenkins labs.

Task	Command with placeholders	Example	What it means
Show current folder	pwd	pwd	Prints the folder path you are currently inside.
List files	ls -la	ls -la	Shows files, hidden files, permissions, owner, size, and date.
Change folder	cd <folder-path>	cd ~/labs/devnet-src/sample-app	Moves your terminal into another folder.
Create folder	mkdir -p <folder>/<subfolder>	mkdir -p sample-app/templates sample-app/static	Creates folders; -p prevents errors if part already exists.
Create empty file	touch <file-name>	touch sample-app.sh	Creates a blank file or updates its timestamp.
View file	cat <file-name>	cat sample_app.py	Prints the file content in the terminal.
Edit file	nano <file-name>	nano Dockerfile	Opens a simple terminal text editor.
Make executable	chmod a+x <script>	chmod a+x sample-app.sh	Allows the script to run with ./script-name.
Run bash script	bash <script-name>	bash sample-app.sh	Runs commands stored inside a shell script.

Tiny trap goblin: Linux is case-sensitive. Dockerfile, dockerfile, and DockerFile are three different names. If Docker says it cannot find a file, check spelling, capital letters, and the folder you are currently in.

Python Basics for DevOps Labs

Python is used in these labs because it is readable and quick to write. In the Docker sample web app, Python works with Flask to create a tiny website. Flask listens for a request, runs a route function, and sends a response back to the browser. When you put this Python app into Docker, the app can run inside a container instead of depending directly on your computer setup.

Python / Flask part	Meaning	Simple student explanation
from flask import Flask	Imports the Flask framework.	This brings in the web app toolbox.
sample = Flask(__name__)	Creates the Flask app object.	This is the app brain.
@sample.route("/")	Defines the home page route.	When someone visits /, this function runs.
request.remote_addr	Gets the visitor IP address.	The app can show where the request came from.
sample.run(host="0.0.0.0",	Runs the app on all network	Using 5050 avoids clashing with

port=5050)	interfaces.	Jenkins on 8080.
------------	-------------	------------------

Module 2 - The DevNet Developer Environment

Cisco DevNet is a developer ecosystem. It gives learners and developers a place to find documentation, tutorials, sandbox environments, sample code, and support. In DevOps terms, DevNet is helpful because it gives you safe places to practice automation before touching real production systems. The most important mindset is this: do not guess how a platform works when its API documentation and sandbox can show you safely.

DevNet resource	What it is	When you use it
Learning Labs	Step-by-step tutorials for APIs, coding, automation, and Cisco technologies.	Use when you need guided practice.
Sandbox	A test environment that behaves like a real platform.	Use when you want to test commands or API calls safely.
Code Exchange	Sample code shared by developers.	Use when you need examples or a starting point.
Developer Documentation	API references and platform guides.	Use when you need exact endpoints, parameters, or authentication details.
Developer Support	Tickets, forums, and support channels.	Use when troubleshooting integrations or API behavior.

Exam vibe: DevNet is not just one website page. It is a collection of learning, testing, documentation, support, and code-sharing tools for developers working with Cisco platforms.

Module 3 - Software Development and Design

Module 3 explains how software is planned and controlled. This is important because DevOps is not only about tools; it is about moving software from idea to users in a safer and faster way. A team needs a development process so work does not become random button-clicking. The process also helps people understand what should be built, who is responsible, how it will be tested, and how changes will be tracked.

Software Development Life Cycle (SDLC)

The SDLC describes the common journey of software from an idea to a working product. The phases are often shown as Requirements and Analysis, Design, Implementation, Testing, Deployment, and Maintenance. In real life, teams may repeat phases or move back and forth, especially in Agile projects. For exam answers, remember that SDLC is bigger than coding because it includes planning, testing, releasing, and supporting the software after release.

SDLC phase	What happens	Easy memory
Requirements and Analysis	The team finds out what problem must be solved and what users need.	Ask: What are we building and why?
Design	The team plans the architecture, components, data, and interfaces.	Draw the blueprint before coding.
Implementation	Developers write the code based on the design.	Build the thing.
Testing	The team checks whether the software works and finds bugs.	Break it safely before users do.
Deployment	The software is released into a production environment.	Send it to the real world.
Maintenance	The team fixes bugs, improves features, and supports users.	Keep it alive and healthy.

Waterfall vs Agile vs Lean

Waterfall is a more linear approach. One phase is usually completed before the next phase starts, so it works best when requirements are stable and clearly known early. Agile is more flexible and uses short cycles so teams can adjust based on feedback. Lean focuses on reducing waste and delivering value to the customer. In exam questions, look for clues: fixed sequence usually points to Waterfall, sprints and user stories point to Agile, and removing waste points to Lean.

Methodology	Main idea	Good for	Watch out
Waterfall	Plan heavily first, then move phase by phase.	Stable projects with clear requirements.	Changes late in the project can be expensive.
Agile	Build in short iterations and keep adjusting.	Projects where feedback and change are expected.	Needs active communication and prioritization.
Lean	Remove waste and maximize customer value.	Teams trying to improve flow and reduce unnecessary work.	Can fail if "waste" is misunderstood or value is unclear.

Agile Scrum Terms

Scrum is one popular Agile method. A sprint is a short fixed period, commonly around two to four weeks, where the team completes selected work. The backlog is the prioritized list of work that could be done. A user story explains a requirement from the user perspective, usually using the pattern "As a user, I want to do something, so that I get a benefit." Daily stand-ups keep the team synchronized without turning into long meetings.

Scrum term	Meaning	Example
Sprint	A short time-boxed work cycle.	Two weeks to build and test a login feature.
Backlog	Prioritized list of features, fixes, and tasks.	Login page, password reset, profile page, bug fix.
User story	Small requirement written from the user point of view.	As a student, I want to reset my password so that I can regain access.
Daily stand-up	Short team sync meeting.	What did I do, what will I do, what is blocking me?
Scrum master	Person who helps remove blockers and keeps Scrum working.	Helps the team stay focused and unblocked.

Design Patterns

Design patterns are reusable solutions to common software design problems. They do not give you finished code, but they give you a proven structure. The Observer pattern is useful when one object needs to notify many other objects about changes, like subscribers receiving updates. MVC splits an app into Model, View, and Controller so the data, display, and input logic are easier to manage separately.

Pattern	Purpose	Simple example
Observer	A subject notifies observers when something changes.	A social media account posts, and followers receive updates.
MVC	Separates data, user interface, and control logic.	A web app separates database data, page display, and button actions.

Version Control and Git

Version control tracks changes to files over time. Git is a distributed version control system, which means each developer can have a full copy of the repository and its history. This helps teams collaborate without overwriting each

other's work. Git uses three important areas: the working directory where you edit files, the staging area where you prepare changes, and the repository where committed snapshots are stored.

Git area/state	Meaning	What command relates to it
Working directory	Visible project files where you make edits.	git status, git diff
Staging area	Selected changes waiting to be committed.	git add <file>
Repository	Stored project history inside .git.	git commit, git log
Modified	File changed but not staged.	git status shows modified.
Staged	File added to staging area.	git add changes the file to staged.
Committed	Snapshot saved in repository history.	git commit saves it.

Task	Command with placeholders	Example	What it means
Set username	git config --global user.name "<name>"	git config --global user.name "Iszy"	Sets the author name for commits.
Set email	git config --global user.email "<email>"	git config --global user.email "iszy@example.com"	Sets the author email for commits.
Start repo	git init	git init	Creates a .git folder and starts tracking history.
Check status	git status	git status	Shows changed, staged, and untracked files.
Stage file	git add <file>	git add sample_app.py	Prepares a file for commit.
Stage all	git add .	git add .	Stages all changes in the current folder.
Commit	git commit -m "<message>"	git commit -m "Add sample app"	Saves staged changes as a snapshot.
View log	git log --oneline	git log --oneline	Shows commit history in a short format.
Create branch	git checkout -b <branch-name>	git checkout -b feature/login-ui	Creates and switches to a new branch.
Switch branch	git checkout <branch-name>	git checkout main	Moves to another branch.
Merge branch	git merge <branch-name>	git merge feature/login-ui	Combines another branch into the current branch.
Add remote	git remote add origin <repo-url>	git remote add origin https://github.com/user/sample-app.git	Connects your local repo to GitHub.
Push	git push -u origin <branch>	git push -u origin main	Uploads commits to GitHub.
Pull	git pull origin <branch>	git pull origin main	Downloads and merges changes from GitHub.
Clone	git clone <repo-url>	git clone https://github.com/user/sample-app.git	Downloads a full copy of a remote repo.

Merge conflict survival: A conflict happens when Git cannot automatically combine changes. Open the conflicted file, look for conflict markers, choose the correct final code, delete the markers, then run git add and git commit. Do not panic; Git is basically asking you to be the referee.

Module 4 - Understanding and Using APIs

An API, or Application Programming Interface, allows one software system to interact with another in a controlled way. Instead of exposing everything inside an application, the API decides what data and actions are available. This is useful for automation because scripts can request information, update systems, or trigger actions without a human clicking through a website. APIs are also important in DevOps because CI/CD tools, cloud platforms, and monitoring systems often communicate through APIs.

API Design Styles

A synchronous API gives a direct response to a request, so the client usually waits until the server replies. This is good when the answer is ready quickly. An asynchronous API accepts the request first and finishes the work later, which is useful when processing takes longer. In simple words, synchronous feels like asking a question and waiting at the counter, while asynchronous feels like taking a queue number and coming back when the order is ready.

Style	How it behaves	Best when	Possible issue
Synchronous	Client sends request and waits for a direct response.	Data is ready quickly and the client needs the answer now.	Client can be blocked while waiting.
Asynchronous	Server confirms request received, then processes later.	Work takes time or data is not immediately available.	Client may need polling, callback, or webhook to know result.

API Architecture Styles

Common API architecture styles include RPC, SOAP, and REST. RPC focuses on calling procedures or methods on another system. SOAP uses XML messages and is often more formal and strict. REST is very common for web APIs because it uses standard HTTP methods and resources. In most C270-style labs, REST is the API style you are most likely to touch directly.

Style	Meaning	Memory hook
RPC	Remote Procedure Call lets one program call a function on another system.	Call a remote action.
SOAP	XML-based messaging protocol with formal structure.	Strict XML envelope.
REST	Resource-based style commonly using HTTP methods.	Work with resources using URLs and verbs.

REST APIs and HTTP Methods

REST APIs usually treat things as resources. A resource might be a user, device, ticket, interface, or order. The URL identifies the resource, while the HTTP method tells the server what action the client wants. GET usually reads data, POST creates something, PUT or PATCH updates something, and DELETE removes something. Status codes help you understand whether the request worked or failed.

Method	Purpose	Example idea
GET	Read or retrieve data.	Get a list of devices.
POST	Create a new resource or submit data.	Create a new ticket.
PUT	Replace or update a resource.	Replace a device configuration.
PATCH	Partially update a resource.	Change only one field.
DELETE	Remove a resource.	Delete a test item.

Status code	Meaning	What to check
-------------	---------	---------------

200 OK	Request succeeded.	The response body usually contains the data.
201 Created	A new resource was created.	Check the new resource ID or location.
400 Bad Request	The request format or parameters are wrong.	Check JSON syntax, missing fields, or wrong values.
401 Unauthorized	Authentication failed or is missing.	Check API key, token, username, or password.
403 Forbidden	Authenticated but not allowed.	Check permissions or role.
404 Not Found	Resource or endpoint does not exist.	Check URL, path, and ID.
429 Too Many Requests	Rate limit was hit.	Slow down requests or wait before retrying.
500 Server Error	Server-side problem.	Check service health or logs if available.

REST Authentication

API authentication proves who or what is making the request. An API key is a simple secret value included with the request. Basic authentication sends a username and password, usually encoded and protected by HTTPS. Bearer tokens are temporary tokens placed in the Authorization header. OAuth is a more complete authorization framework often used when an app needs controlled access without sharing the user's password directly.

Authentication type	How it works	Watch out
API key	Client sends a secret key with the request.	Easy to use but must be protected like a password.
Basic auth	Client sends username and password.	Should only be used over HTTPS.
Bearer token	Client sends a token in the Authorization header.	Tokens can expire and need refresh.
OAuth	Uses authorization flow and tokens for delegated access.	More secure and flexible, but more complex.

Task	Command with placeholders	Example	What it means
GET request	<code>curl -X GET <url></code>	<code>curl -X GET https://api.example.com/devices</code>	Sends a read request to an API endpoint.
GET with token	<code>curl -H "Authorization: Bearer <token>" <url></code>	<code>curl -H "Authorization: Bearer abc123" https://api.example.com/devices</code>	Adds a bearer token header.
POST JSON	<code>curl -X POST <url> -H "Content-Type: application/json" -d '<json>'</code>	<code>curl -X POST https://api.example.com/tickets -H "Content-Type: application/json" -d '{"title": "test"}'</code>	Sends JSON data to create or submit something.
Show headers	<code>curl -i <url></code>	<code>curl -i https://api.example.com/status</code>	Displays response headers and status code.

Webhooks and Rate Limits

A webhook lets one system send a message to another system when an event happens. Instead of repeatedly asking “anything new yet?”, the receiving system waits for the event notification. Rate limits prevent clients from sending too many API calls too quickly. They protect the service from overload and encourage developers to write efficient code.

API troubleshooting order: Check the URL, HTTP method, headers, authentication, request body, and status code. If the status code is 401, think authentication. If it is 403, think permission. If it is 404, think wrong endpoint or resource ID. If it is 429, slow down your requests.

Module 6 - Application Deployment and Security

Module 6 is where the DevOps pieces become more practical. Developers do not only write code; they also need to think about where the application runs, how it is packaged, how it is tested, how it is deployed, and how it is secured. This is why the module includes deployment models, Docker containers, CI/CD, network needs, and application security. The main idea is that software should move through environments in a controlled and repeatable way.

Deployment Environments

Many teams use separate environments so unfinished or risky code does not go straight to real users. Development is where code is written. Testing is where the team checks whether the code works. Staging is a production-like environment used for final checks. Production is the real environment used by customers. Keeping these environments separate reduces risk and makes troubleshooting easier.

Environment	Purpose	Student memory
Development	Where developers write and run early code.	Messy workshop.
Testing	Where the team verifies features and catches bugs.	Safe bug-hunting zone.
Staging	Production-like final check environment.	Dress rehearsal.
Production	Live system used by real users.	Real show, no clown shoes.

Deployment Models

Applications can be deployed in several ways. Bare metal means software runs directly on a physical machine. Virtual machines share hardware but have their own guest operating systems. Containers share the host operating system kernel and package the app with its dependencies. Serverless lets developers run functions or services without managing servers directly, and it usually charges based on actual use.

Model	Meaning	Pros	Cons / Watch out
Bare metal	Software runs directly on physical hardware.	Strong control and performance.	Harder to scale and maintain.
Virtual machine	A software computer with its own OS.	Good isolation and familiar server model.	More overhead than containers.
Container	Packaged app sharing the host kernel.	Fast startup and consistent deployment.	Needs image/container knowledge.
Serverless	Provider runs code when needed.	Less server management and pay-per-use.	Less control over runtime details.

Infrastructure Types

On-premises infrastructure is owned and managed inside an organization's physical location. Private cloud gives cloud-like self-service but under one organization's control. Public cloud is managed by a provider and can scale quickly. Hybrid cloud combines private and public resources. Edge cloud places resources closer to users or devices, which can reduce latency for IoT and location-sensitive workloads.

Infrastructure	Meaning	Best clue
On-premises	Systems physically located in the organization.	Full control, but slower scaling.

Private cloud	Cloud-like environment controlled by one organization.	Control and compliance.
Public cloud	Cloud services from a provider shared across customers.	Fast scaling and less hardware ownership.
Hybrid cloud	Mix of private and public cloud resources.	Keep sensitive parts private, burst or expand in public cloud.
Edge cloud	Compute closer to users or devices.	IoT, low latency, local processing.

Docker Concepts

Docker is used to containerize applications. A Docker image is a read-only package that contains the application code, runtime, libraries, and dependencies. A Docker container is the running instance of that image. A Dockerfile is the recipe used to build the image. This matters because containers make applications easier to move between development, testing, staging, and production.

Docker term	Meaning	Example
Dockerfile	Text file with instructions to build an image.	FROM python, RUN pip install flask, COPY files.
Image	Read-only packaged application template.	sampleapp:latest.
Container	Running instance of an image.	samplerunning.
Registry	Place where images are stored and shared.	Docker Hub or private registry.
Port mapping	Connects host port to container port.	-p 5050:5050.

Task	Command with placeholders	Example	What it means
Check Docker	docker --version && docker ps	docker --version && docker ps	Confirms Docker is installed and running.
Build image	docker build -t <image-name> <path>	docker build -t sampleapp .	Builds an image from a Dockerfile.
Run container	docker run -d -p <host-port>:<container-port> --name <container-name> <image-name>	docker run -d -p 5050:5050 --name samplerunning sampleapp	Runs a container in the background with port mapping.
List containers	docker ps -a	docker ps -a	Shows running and stopped containers.
Stop container	docker stop <container-name>	docker stop samplerunning	Stops a running container.
Remove container	docker rm <container-name>	docker rm samplerunning	Deletes a stopped container.
View logs	docker logs <container-name>	docker logs samplerunning	Shows output from the container.
Remove image	docker rmi <image-name>	docker rmi sampleapp	Deletes an image if no container uses it.
Inspect network	docker network inspect bridge	docker network inspect bridge grep Gateway	Finds Docker network details such as gateway IP.

Dockerfile not found fix: Run `ls -la` and `pwd`. Make sure the Dockerfile name matches exactly and you are inside the folder containing it. If the command uses `-f Dockerfile.frontend`, then that exact file must exist or Docker will fail.

CI/CD and Jenkins

Continuous Integration means developers frequently merge code into a shared repository, where automated builds and tests can run. Continuous Delivery means the code is kept in a deployable state after passing checks. Continuous Deployment goes one step further by automatically deploying changes after the pipeline passes. Jenkins is an automation server that can run jobs and pipelines for build, test, and deploy steps.

Term	Meaning	Why it matters
CI	Automatically build and test code after changes.	Finds problems earlier.
CD - Delivery	Keeps software ready to deploy.	Release is safer and more predictable.
CD - Deployment	Automatically deploys passing changes.	Fast release flow, but needs strong testing.
Jenkins job	A configured automation task.	Can build or test the app.
Pipeline	A chain of stages such as build, test, and deploy.	Makes the process repeatable.

Task	Command with placeholders	Example	What it means
Pull Jenkins image	<code>docker pull jenkins/jenkins:its</code>	<code>docker pull jenkins/jenkins:its</code>	Downloads the Jenkins image.
Run Jenkins	<code>docker run --rm -u root -p 8080:8080 -v jenkins-data:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock --name <name> jenkins/jenkins:its</code>	<code>docker run --rm -u root -p 8080:8080 -v jenkins-data:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock --name jenkins_server jenkins/jenkins:its</code>	Starts Jenkins on port 8080 and allows Docker access.
Get admin password	<code>docker exec -it <jenkins-container> cat /var/jenkins_home/secrets/initialAdminPassword</code>	<code>docker exec -it jenkins_server cat /var/jenkins_home/secrets/initialAdminPassword</code>	Prints the first Jenkins login password.
Stop Jenkins	<code>docker stop <jenkins-container></code>	<code>docker stop jenkins_server</code>	Stops the Jenkins container.

A simple Jenkins lab often has one job to build the app and another job to test it. The build job can pull code from GitHub and run a script such as `sample-app.sh`. The test job can run a `curl` command to verify that the app responds correctly. A pipeline links these stages together so one successful step can trigger the next. If one stage fails, the pipeline stops and the console output becomes your treasure map.

Jenkins port warning: Jenkins commonly uses port 8080. If your Flask sample app also uses 8080, the two services will fight for the same port. Change the Flask app to port 5050 or another free port, then update Dockerfile, scripts, and test commands to match.

Lab 3.3.11 - Software Version Control with Git

This lab teaches the Git life cycle by having you create a repository, make a file, stage it, commit it, modify it, branch it, merge it, and connect it to GitHub. The important skill is not just copying commands. You should understand what state your files are in after each command. When something goes wrong, `git status` is usually the first command to run because it tells you what Git sees.

1. Configure your Git name and email so commits show who made them.

2. Create or enter a project folder and run `git init` to start a repository.
3. Create or edit files in the working directory.
4. Run `git status` to see what changed.
5. Run `git add` to stage the files you want in the next snapshot.
6. Run `git commit -m "message"` to save the snapshot.
7. Use branches when you want to work safely without touching main code.
8. Push to GitHub when you want your local commits stored remotely.

Git lab troubleshooting: If Git says “nothing to commit,” your changes may already be committed or you may be in the wrong folder. If Git says “repository not found,” check the remote URL, GitHub username spelling, repository name, and whether you have permission.

Lab 6.2.7 - Build a Sample Web App in a Docker Container

This lab shows how a simple Python Flask app can be packaged into a Docker container. You create website files, create a shell script, generate or use a Dockerfile, build an image, and run a container. The main learning point is repeatability. Instead of manually installing Flask and running Python every time, the Docker build captures the setup steps so the app can run in a consistent environment.

9. Create the project folders for templates and static files.
10. Write `sample_app.py` so Flask can respond to browser requests.
11. Create `index.html` and `style.css` so the page has structure and styling.
12. Create `sample-app.sh` to automate copying files, building the image, and running the container.
13. Run the script and confirm the container is listening on the correct port.
14. Use `docker ps -a`, `docker logs`, `curl`, and the browser to verify the app.

Docker lab troubleshooting: If the web page does not open, check that the container is running, the port is mapped correctly, and the Flask app is using the same port as the docker run command. If the container name already exists, stop and remove the old container before running again.

Lab 6.3.6 - Build a CI/CD Pipeline Using Jenkins

This lab connects GitHub, Docker, and Jenkins into a small CI/CD workflow. GitHub stores the code. Jenkins pulls the code and runs build commands. Docker packages and runs the application. A test job checks whether the application responds correctly. The full pipeline gives you a repeatable flow from code change to build and test result.

15. Create or use a GitHub repository for the sample app files.
16. Push the app files to GitHub so Jenkins can access them.
17. Run Jenkins in a Docker container and unlock it with the initial admin password.
18. Create a build job that pulls from GitHub and runs the build script.
19. Create a test job that checks the app with `curl`.
20. Create a pipeline so build and test steps happen in sequence.
21. Read Console Output when a job fails because that is where Jenkins explains the problem.

Common Jenkins problem	Likely cause	Fix
Cannot clone GitHub repo	Wrong URL, private repo, or missing PAT credentials.	Check URL, branch, username, and token.
Build fails on Docker command	Jenkins container cannot access Docker socket or binary.	Check Docker mount options when running Jenkins.
Test job cannot curl app	Wrong IP, wrong port, or app container not running.	Use <code>docker ps -a</code> and <code>docker network inspect bridge</code> .
Port already in use	Jenkins and Flask both using 8080 or old container still running.	Use port 5050 for Flask and remove old containers.

Branch not found	Jenkins job uses */master but repo uses main, or the reverse.	Set branch specifier to the real branch name.
------------------	---	---

Troubleshooting Mini-Guide

Troubleshooting is basically detective work with receipts. Do not guess too quickly. Read the error message, check the folder, check the file names, check whether the service is running, then check logs. Most lab problems come from being in the wrong directory, using the wrong port, using the wrong branch name, or missing a file that the command expects.

Symptom	Most likely reason	Commands to try
No such file or directory	Wrong folder or wrong file name.	pwd, ls -la, find . -name "<file>"
Dockerfile not found	Docker build path or -f filename is wrong.	pwd, ls -la, docker build -f <Dockerfile-name> -t <image> .
Port already allocated	Another container or service is using the port.	docker ps -a, docker stop <container>, docker rm <container>
Container exits immediately	App crashed or command ended.	docker logs <container>, docker inspect <container>
Git push rejected	Remote has changes you do not have locally.	git pull origin <branch>, resolve conflicts, git push
Authentication failed	Wrong username/password/token or expired token.	Check PAT, credentials, scopes, and repository access.
Jenkins build failed	Script error, Git error, Docker error, or wrong branch.	Open Console Output and read from the first red/error line.

Final Command Cheat Sheet

This section is for quick exam or lab use. The placeholders are the parts that may change. Replace anything inside angle brackets with your own value. For example, replace <branch> with main, master, or the branch your teacher gives you.

Linux and Files

Task	Command with placeholders	Example	What it means
Where am I?	pwd	pwd	Shows current directory.
Show files	ls -la	ls -la	Shows files and permissions.
Go to folder	cd <path>	cd ~/sample-app	Changes directory.
Create folders	mkdir -p <folder>/<subfolder>	mkdir -p app/templates/app/static	Creates folders safely.
Edit file	nano <file>	nano sample_app.py	Opens terminal editor.
Run script	bash <script>	bash sample-app.sh	Runs bash script.

Git

Task	Command with placeholders	Example	What it means
Initialize repo	git init	git init	Starts Git tracking.
Check changes	git status	git status	Shows current file state.
Stage changes	git add <file-or-.>	git add .	Prepares changes for commit.
Commit changes	git commit -m "<message>"	git commit -m "Update app"	Saves snapshot.

Add remote	git remote add origin <repo-url>	git remote add origin https://github.com/user/repo.git	Connects to GitHub.
Push branch	git push -u origin <branch>	git push -u origin main	Uploads commits.
Pull branch	git pull origin <branch>	git pull origin main	Downloads and merges remote changes.
New branch	git checkout -b <branch>	git checkout -b feature/homepage	Creates and switches branch.

Docker

Task	Command with placeholders	Example	What it means
Build image	docker build -t <image> <path>	docker build -t sampleapp .	Builds image.
Run container	docker run -d -p <host>:<container> --name <name> <image>	docker run -d -p 5050:5050 --name samplerunning sampleapp	Runs container.
List containers	docker ps -a	docker ps -a	Shows containers.
Stop container	docker stop <name>	docker stop samplerunning	Stops container.
Remove container	docker rm <name>	docker rm samplerunning	Deletes stopped container.
View logs	docker logs <name>	docker logs samplerunning	Shows container output.

Jenkins

Task	Command with placeholders	Example	What it means
Pull Jenkins	docker pull jenkins/jenkins:its	docker pull jenkins/jenkins:its	Downloads Jenkins image.
Run Jenkins	docker run --rm -u root -p 8080:8080 -v jenkins-data:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock --name <name> jenkins/jenkins:its	docker run --rm -u root -p 8080:8080 -v jenkins-data:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock --name jenkins_server jenkins/jenkins:its	Starts Jenkins.
Get password	docker exec -it <name> cat /var/jenkins_home/secrets/initialAdminPassword	docker exec -it jenkins_server cat /var/jenkins_home/secrets/initialAdminPassword	Gets unlock password.

One-Page Memory Hook

Think of DevOps as a delivery kitchen. The SDLC is the recipe plan. Git is the notebook that records every recipe change. GitHub is the shared kitchen shelf where the team stores the notebook. Docker is the lunchbox that packs the app with everything it needs. Jenkins is the conveyor belt that builds, tests, and moves the lunchbox. APIs are the ordering window between systems. Security is the bouncer making sure only the right people and apps can get in. Tiny goblin summary: plan it, track it, package it, test it, deploy it, secure it.